7 August 2025

End-to-End Threat Hunting with a Novel Multiclass Dataset for Intelligent Intrusion Detection

Qasem Abu Al-Haija[1], Zaid Masoud[2], Assim Yasin[2], Karam Alesawi[2], Yousef Alkarnawi[2]

1. Cybersecurity Department, Jordan University of Science and Technology, Irbid, Jordan

2. Cybersecurity Department, Princess Sumaya University for Technology

Abstract

With the rapid evolution of technology, interconnected networks have become essential across all digital systems, enabling seamless global integration. While these advancements enhance system capabilities and user experiences, they also introduce significant cybersecurity risks. We propose a comprehensive threathunting framework spanning data generation to intelligent intrusion detection to address this. The objective is to design a robust intrusion detection system (IDS) capable of distinguishing normal from malicious traffic and classifying threats into 15 distinct cyberattack categories, including MITM ARP Spoofing, SSH Brute Force, SQL Injection, and DDoS variants. Our approach introduces a novel, largescale multiclass dataset for modern network environments. We evaluated this dataset using seven supervised learning models: Naïve Bayes (NB), Logistic Regression (LR), Random Forest (RF), Decision Trees (DT), Feedforward Neural Networks (FNN), Multi-Layer Perceptron (MLP), Deep Convolutional Neural Networks (DCNN), and Shallow Convolutional Neural Networks (SCNN). Among these, DT-based models demonstrated the highest performance, achieving 99.9% accuracy in detection and classification, with a minimal prediction latency of 1.1 microseconds. This research contributes a scalable and effective IDS framework to enhance the security posture of communication networks against diverse and evolving cyber threats.

Keywords

anomaly detection (ad), communication, networking and broadcast technologies, computing and processing, cybersecurity, deep learning (dl), intrusion detection system (ids), machine learning (ml), network traffic analysis, threat classification

Posted on 7 August 2025 — CC-BY 4.0 — This is a preprint and has not been peer reviewed. Data may be preliminary. — https://doi.org/10.36227/techrxiv.175459955.56010835/v1

End-to-End Threat Hunting with a Novel Multiclass Dataset for Intelligent Intrusion Detection

Qasem Abu Al-Haija [1,*], Zaid Masoud [2], Assim Yasin [2], Karam Alesawi [2], Yousef Alkarnawi [2]
1 Cybersecurity Department, Jordan University of Science and Technology, Irbid, Jordan
2 Cybersecurity Department, Princess Sumaya University for Technology, Amman, Jordan
* Corresponding Author: qsabuhaija@just.edu.jo

Abstract: ***With the rapid evolution of technology, interconnected networks have become essential across all digital systems, enabling seamless global integration. While these advancements enhance system capabilities and user experiences, they also introduce significant cybersecurity risks. We propose a comprehensive threat-hunting framework spanning data generation to intelligent intrusion detection to address this. The objective is to design a robust intrusion detection system (IDS) capable of distinguishing normal from malicious traffic and classifying threats into 15 distinct cyberattack categories, including MITM ARP Spoofing, SSH Brute Force, SQL Injection, and DDoS variants. Our approach introduces a novel, large-scale multiclass dataset for modern network environments. We evaluated this dataset using seven supervised learning models: Naïve Bayes (NB), Logistic Regression (LR), Random Forest (RF), Decision Trees (DT), Feedforward Neural Networks (FNN), Multi-Layer Perceptron (MLP), Deep Convolutional Neural Networks (DCNN), and Shallow Convolutional Neural Networks (SCNN). Among these, DT-based models demonstrated the highest performance, achieving 99.9% accuracy in detection and classification, with a minimal prediction latency of 1.1 microseconds. This research contributes a scalable and effective IDS framework to enhance the security posture of communication networks against diverse and evolving cyber threats***.

Keywords—Intrusion Detection System (IDS), Cybersecurity, Network Traffic Analysis, Machine Learning (ML), Deep Learning (DL), Threat Classification, Anomaly Detection (AD).

I. INTRODUCTION

As advancements in the digital world continue to increase rapidly, the robustness of network security is becoming a significant concern. This traces back to the extremely high growth in data density and exposure to new advanced cybersecurity-related threats and risks. These advanced threats, including zero-day attacks and polymorphic malware, have created a big challenge for classic rule-based security tools and methods [1]. This poses the need for a shift in the approaches used to achieve network security.

After recognizing this issue, this project creates a comprehensive approach for innovating solutions, specifically deploying ML techniques to improve network security. This project begins by developing and refining new methodologies for generating and preprocessing network traffic using the capabilities of Linux, pfSense, and other powerful tools. Data is monitored through Ubuntu to achieve a strategic approach towards network traffic management. Another valuable tool for this project is OWASP, the Open Web Application Security Project, aiding legitimate vulnerability testing. Furthermore, this project expands its limits by performing controlled external and internal attacks simulated through another Linux machine operated by Kali. This research aims to build a reliable dataset to enhance the dynamic training of ML models. We build these models to recognize various categories of network threats, enabling them to adapt and grow in response to the rapid advancements in cybersecurity. The eventual objective is to automate the process of abnormal traffic detection, enabling a new implementation that provides real-time responses and ultimately improves network security. The result of a fully interconnected world, where there is high reliability on systems that seem simple from the outside but are much more complex for communication, exchanges, and data storage, is that the demand for new contemporary security frameworks has increased. This project seeks to achieve essential goals regarding network security. This project aims to provide adaptable solutions that not only counter today's threats but also mitigate the upcoming challenges of the future.

A. Problem Statement

This project addresses emerging cybersecurity concerns as conventional measures weaken and fail to secure network systems. Due to intensive data roaming across networks, additional hazards have developed [2]. This solves the demand for innovative, strong network security methods. This project produced a sophisticated ML-based network security system. The system's real-time monitoring of unusual network activities will distinguish it. The project will also produce advanced ML models, a data preprocessing framework, and network monitoring tool integration. In addition to threat modeling, data generation for these models is difficult. Real-world, diversified datasets, ethical considerations while exploiting the attack data block, and robust model development and validation are needed [3]. A complete and reliable methodology for simulating network threats requires closing the gap between created and real-world data sources. This effort targets data transmission security-dependent companies and networks. This system's end users include security personnel, data administrators, and others seeking digital infrastructure integrity. This project gives the targeted audience an up-to-date and automated network security process, lowering security administrators' effort, and providing high-level security protection for sensitive data in the interconnected world. This research empowers enterprises to defend their internal and external networks and vital data from cyberattacks.

B. Significance of the study

This research may change and elevate network security in an age of digital connectivity. Rule-based detection approaches have failed to adapt to cyberthreats' innovation [4]. Using advanced ML algorithms is now necessary. This project tries to improve network security by using ML for real-time detection and response to suspect network activity to improve the security of the organization and networks. A rich dataset with different network situations and security occurrences is essential for

training and fine-tuning ML models [5]. This dataset enhances scholarly understanding of network vulnerabilities and has real-world applications. Practicality is added to the research by deploying Ubuntu Linux, pfSense, the OWASP Broken Web App, and Kali Linux for testing. The research tries to bridge theoretical constructs and actual application by demonstrating the viability and applicability of proposed approaches in real-world scenarios. The findings of this study can help firms automate identifying suspicious network activity, enabling quick and effective responses to threats. By doing so, the initiative hopes to make a meaningful and enduring contribution to protecting networked systems and their important data in our increasingly interconnected world.

C. Study objectives

In this project, our main goal is to develop an inclusive and trustworthy threat-hunting model, covering the entire spectrum from data generation to intelligent threat detection. Specifically, the key objectives of this project can be summarized as follows:

- Developing a comprehensive understanding of existing network attack modeling approaches and their limitations.
- Exploring various techniques to generate synthetic and real data, which will enrich the training datasets for attack models.
- Investigating the application of intelligent threat detection mechanisms, focusing on ML and DL algorithms.
- Proposing an integrated framework that addresses the challenges of both data generation and intelligent threat detection in the context of network security.
- Evaluating the Efficacy of Current Network Defense Strategies and the robustness of the adversarial attacks framework.

D. Study Contribution

- **Novelty in the idea:** The project introduces a groundbreaking approach to network security by integrating ML into traditional defense mechanisms. The novelty lies in the proactive and adaptive nature of the proposed system, which goes beyond conventional rule-based methods. Leveraging Ubuntu Linux, pfSense, and OWASP Broken Web App for realistic testing and intelligent threat detection represents a pioneering concept in fortifying network security.
- **The audience that it serves and how:** The primary audience for this project includes network administrators, cybersecurity professionals, and organizations seeking robust security measures. The project serves them by offering an advanced network security system that automates threat detection and response. The system's user-friendly interfaces and integration with existing tools make it accessible to technical and non-technical users. Its impact is evident in reducing the workload on administrators while significantly enhancing the security posture of organizations in the face of evolving cyber threats.
- **New approach to utilizing ML models for threat detection**: By exploring and fine-tuning a diverse set of ML and DL algorithms, the research ensures the selection of models tailored to the intricacies of network security. This flexibility & adaptability distinguish the work from conventional approaches, providing a cutting-edge solution that aligns with the dynamic nature of cyber threats.
- **Novelty in the pipeline structure:** The project contributes novelty to the pipeline structure by orchestrating a seamless integration of components. Using Ubuntu Linux and pfSense for real-world monitoring and the OWASP Broken Web App and Kali Linux for testing demonstrates a holistic and practical approach. The pipeline's structure is designed for scalability and adaptability, ensuring its effectiveness in various network environments. Integrating data generation techniques with intelligent threat detection forms a novel & cohesive pipeline, addressing data quality challenges & real-time threat response.

II. LITERARY REVIEW

This section evaluates several methodologies, data generation methods, and techniques related to intelligent threat detection in the network security landscape. The review compares the presented studies and our project's approach. Additionally, it recognizes and presents all challenges, similarities, differences, and advantages deployed to improve our research's effectiveness.

Enhanced anomaly detection (AD) with hybrid models [7]:

In this research, the authors explored the utilization and deployment of AD. Similarly, our work grasps the advantages & strong points of CNNs, integrating several ML techniques. This helps achieve better results with more flexibility and robustness. In particular, this creates effective management of complexities for networks. Our model will provide a more practical solution that can adapt to several scenarios and has a higher AD capability than single-method systems, which could misclassify.

Advanced IoT security through diverse data analysis [8]: Pandeeswari et al provided helpful information about the ML landscape for IoT security. However, our research crosses the limitations by adopting a wider range of data with multiple types of network activities that generate it. We implemented advanced data preprocessing methods to improve data quality and our system's detection accuracy. This approach to analyzing data helps our system prevent actual threats from being processed. If deployed in a specific manner, our system can significantly impact the security of IoT.

Integrated learning methods against modern cyber threats [9]: Goel et al. developed a crucial approach utilizing conventional ML techniques for intrusion detection**.** Although this was a powerful measure, our project, on the other hand, identified the rapidly advancing nature of cybersecurity-related threats and acknowledged them by merging traditional techniques with more advanced DL techniques. This has led to a more sophisticated detection mechanism, identifying patterns, not only signatures, of advanced cybersecurity-related threats.

Adaptable DL for varied network security scenarios [10]:

Sezari et al. worked on securing networks interconnected with airport security. This was achieved by Sezari et al. with the use of deep feedforward networks. We built upon Sezari et al.'s concept by applying DL models to become adaptable for several other network environments, not just airport security. This helps achieve a universally applicable model effective in different setups, from organizational networks to public systems.

Optimized ML for enhanced security performance [11], [12]:

Rahman et al. have emphasized optimizing ML algorithms, underscoring the importance of bespoke solutions in network security. Our research differs by enhancing these algorithms to achieve higher detection rates and incorporating real-time data processing capabilities. This ensures that our system maintains high speed and accuracy flexibly, an essential combination to provide a real-time response to threats.

Robust datasets utilize comprehensive security [13], [14]. These two studies focused on dataset generation for IoT and graph-based DL models, providing a new perspective on the importance of diverse data sources and innovative modeling techniques. Our research accommodates these elements to employ a range of datasets and encompasses traditional network data, IoT-specific information, and synthetic data for training. Additionally, graph-based models improve our system's ability to decipher complex patterns and relationships in network traffic and provide a different understanding of possible risks.

Dynamic IDS using advanced learning techniques [15], [16]:

The authors investigated DL models on the Apache Spark platform, scrutinizing optimized ML algorithms such as KNN and SVM, emphasizing the importance of dynamically responsive systems for achieving network security. We outperformed these studies by enhancing these algorithms, ensuring their compatibility with real-time data analysis. This combination of methods and techniques made our system quickly adjust to continuously changing networks and new threats, maintaining a high level of security for these networks.

AI-Powered threat detection & predictive security [17],[18]: the authors looked at how AI helps find network threats, using techniques like bagging ensemble methods with RF classifiers, to explore the potential of AI in the cybersecurity field. This is where our research differs: using AI, explicitly focusing on ML algorithms' capabilities in learning from dense datasets to predict and provide a preemptive response to and eliminate potential threats. Our system's ability to process large and dense data led to our research in proactive network security.

Innovative firewall integration & anomaly detection [19], [20]:

Integrating ML side by side with firewalls and exploring models such as Isolation Forest (IF) and SVM for AD clarifies significant enhancements in network security. We complemented these advancements by developing systems that merge smoothly with existing firewall infrastructures to enhance their capabilities through advanced AD techniques. Following this dual approach ensures that our system is not just an 'add-on' but a significant enhancement to existing security measures, thus providing a robust defense mechanism that consists of even more layers.

DDoS attacks and network traffic prediction [21], [22]:

These focused on analyzing network traffic to detect DDoS attacks using RNNs to predict traffic, highlighting how quickly network threats are evolving and the necessity for new security solutions. This is where employing advanced data analysis techniques and optimizing RNNs becomes handy. This method helps our system spot and reduce DDoS attacks effectively while also helping us predict traffic patterns, which allows us to take steps to prevent possible issues.

Novel ML techniques and synthetic data utilization [23], [24]: These employed ML techniques for recent network-based attacks and generated synthetic network traffic datasets to improve training in cybersecurity. Our research approach integrates proven and unconventional ML techniques, combining real and synthetic data. This approach ensures that our system is continuously trained on actual scenarios and is armed with the most advanced tools and techniques to battle emerging threats effectively.

Specialized IoT and networking malware detection [25], [26]: These studies utilized specific IoT networking malware detection analysis to provide well-established approaches to network security. We exceeded these two types of research by taking a more integrated approach and designing a functional system to handle IoT-specific threats and broader network challenges. Combining regression analysis with advanced ML techniques helped make our system dynamic and responsive in a real-time manner, providing the capability of adapting to several settings.

Advanced analytics for network packet identification [27]:

In this work, the authors focused on recognizing abnormal network packets through advanced analytics, which raises the need for precise and complex detection mechanisms. In this area, we have made advancements by incorporating real-time data analysis and adaptive algorithms, which enhance the system's ability to detect and respond to abnormal packets rapidly. This rapid response is essential to prevent possible breaches and maintain network integrity.

Leveraging distributed DL for scalable solutions [28]:

Here, the researchers explored distributed DL methods for network data analysis, raising the challenge of managing highly dense data processing. Leveraging these methods while simplifying the architecture to ensure efficiency and manageability is a solution. This balance is essential to maintaining performance in an extensive network environment, thus making our system more powerful and practically efficient for several applications.

Predictive next-generation cyberattack prevention system [29]: The authors discussed predictive techniques for potentially dangerous cyberattacks. These techniques mainly consist of regression algorithms. Therefore, we decided to implement a system that merges ML algorithms capable of processing massive datasets. This led to great success anticipating future cyber threats, putting it at the leading edge of next-generation cyberattack prevention.

Real-Time anomaly detector in network traffic [30]:

This study emphasized real-time AD in network traffic, addressing the necessity of rapid data processing in cybersecurity. Our research optimizes ML models to achieve speed and efficiency, which helps us ensure that the density or complexity of the data does not compromise real-time AD. This capability is essential to our project's effectiveness in speedily identifying and mitigating potential threats.

The integration of DL & ML methods [7, 9, 10, 14, 15, 16, 28]:

These researchers demonstrated the trend towards combining DL with traditional ML techniques and methods. This is called the hybrid approach. We reflected on the hybrid approach by harnessing the strengths of both methodologies to create a more unique and effective security system.

Real-time processing/predictive abilities [12, 15, 16, 22, 29, 30]: These research studies highlighted the importance of real-time data processing and predictive analysis in network security. The method taken in our research aligns with this focus to optimize real-time threat detection & employ predictive algorithms to foresee future threats.

Advanced data analysis & AI techniques [11, 17, 18, 21, 23, 24, 27]: These research studies highlighted using advanced data analysis & AI techniques to improve network security**.** Modern data analysis techniques & AI algorithms align perfectly with these

advancements to ensure that the organization remains in a leading position in cybersecurity technology.

Adaptability to various network environments [8, 10, 13, 19, 25, 26]: These research studies indicated the growing need for security solutions to be highly adaptable to different network environments & settings. Versatile design & flexible architecture address this need, thus making it perfect for several applications anywhere, from IoT security to substantial network protection.

IDS-based SVM-based DL for improved network security [31]: Abuali et al. presented a hybrid IDS using a combination of support vector machines and DL techniques. This research is relevant because it focuses on classifying data extracted from servers for intrusion incidents in social media networks and aims to achieve secure networks, including social media networks.

HSDL-based intelligent IDS framework for IoT network [32]: This study focused on IoT networking security utilizing a hybrid stacked DL model that combines a vectorized CNN and a stacked long short-term memory (LSTM) network, offering insights into effectively handling and classifying IoT network traffic data. This methodology begins by addressing IoT security challenges, including data preprocessing and real-time threat detection. It aligns with the goal of this paper, which is to enhance IoT network security. Their successful application of DL techniques in detecting threats in diverse IoT scenarios provides a valuable framework for our research's progress.

Improve network security using advanced port scanning [33]: Abu Bakar et al. concentrated on utilizing DPDK-based scanners for effective network security. This study aligns with our research objectives by demonstrating improved port scanning methods, leveraging smart NICs, and using DPDK frameworks to enhance packet processing. Their scanning speed, accuracy, and resource utilization findings offer crucial insights, and adapting these techniques significantly benefits our project, focusing on network security enhancements & efficient network management.

OD1NF1ST: IDS & avionics network attack simulation [34]: The paper presented an innovative approach to enhancing the security of MIL-STD-1553 communication buses in military avionics. The paper introduces a software system for simulating bus transmissions, creating datasets of MIL-STD-1553 communications, and a large IDS that utilizes recurrent neural networks with a reinforcement learning true-skip data selection algorithm. This IDS explains improved performance in detecting and classifying cyberattacks to optimize computational resources and integrate attack scenarios in detail compared to existing algorithms. The research contributes substantially to the field of cybersecurity in avionics networks.

Analyzing TOR artifacts in virtualized Windows 11 [35]: The authors investigated the TOR (The Onion Router) network's use in Windows 11 by examining TOR browser artifacts, their behavior in Windows 11, and the digital evidence they leave behind. The paper simulates TOR network traffic and captures network forensics artifacts. It aims to create a dataset for ML models to detect TOR traffic and help clarify TOR browser usage and associated security risks; this aligns with our research's focus on network traffic analysis and generating various datasets for intelligent threat detection.

Intelligent system to identify & classify malicious URLs [36]: This paper proposed a high-performance machine learning-based system for detecting and classifying malicious URLs. The system operates on two levels: initially, it identifies URLs as benign/malware using a binary classifier and subsequently classifies them into five categories: benign, spam, phishing, malware, and defacement. The study reports and evaluates four ensemble learning approaches on the ISCX-URL2016 dataset, providing outcomes highlighting these models' effectiveness in accurately detecting and categorizing malicious URLs with the help of the bagging trees (En_Bag) ensemble approach, demonstrating superior performance. This research is significant for network security, specifically in highlighting and mitigating cyber threats. The methodology involving ensemble learning approaches for threat detection is similar to our project's emphasis on using advanced ML techniques for improving network security. To sum up, Table 1 provides a comparative summary of all the studies surveyed.

Table 1: Summary of related research

No.	Framework	Main Idea	Model/s	Goals	Domain
[7]	CNN	CNN will detect abnormal network traffic using the MAWILab dataset.	CNN	Detect and analyze network anomalies.	Network Traffic Analysis
[8]	Analysis of ML Techniques	ML techniques to detect intrusions.	Multiple ML	Assess the effectiveness of different ML techniques in IDS.	Cybersecurity
[9]	Supervised ML Techniques	AD using supervised learning methods.	Supervised ML	Detecting network attacks	Network Security
[10]	DL Models	Airport systems use the DL model for network intrusion detection.	DL Models	Secure airport network systems from intrusions.	Airport Network Security
[11]	ML Algorithms on MiniVHS-22 Dataset	The MiniVHS-22 dataset employs ML algorithms for IDS.	Various ML Algorithms	Improve threat detection in diverse network environments.	Cybersecurity
[12]	Optimization Techniques for ML Algorithms	Optimized ML algorithms for efficient intrusion detection.	Various ML Algorithms	Enhance accuracy and efficiency in intrusion detection.	Computer Network Security
[13]	Dataset Generation for IoT Security	The framework creates datasets to assess IoT Linux host-based IDS.	Dataset Generation	Facilitate research in IoT security.	IoT Security
[14]	Graph-Based DL	The system uses graph-based DL for attack detection in network traffic.	Graph-Based DL Models	Detect attacks in network traffic using advanced techniques.	Network Traffic Analysis
[15]	DL on Apache Spark	DL models for IDS are implemented using Apache Spark.	DL Models on Apache Spark	Optimize intrusion detection in terms of accuracy and efficiency.	Computer Network Security
[16]	Optimization Techniques for ML Algorithms	Optimized ML algorithms for efficient intrusion detection.	Various ML Algorithms	Enhance accuracy and efficiency in intrusion detection.	Computer Network Security
[17]	Machine Learning	Developing a network threat detection system using ML algorithms.	LightGBM, RF, KNN	The system is highly accurate in identifying network threats.	Network Security

[18]	Bagging Ensemble with RF	Improving network traffic analysis accuracy for 5G networks.	BRF Technique	Enhanced accuracy in traffic classification.	5G Network Traffic Analysis
[19]	Machine Learning, DL	Improving firewalls' detection capabilities via ML and DL techniques.	LR, CNN	Enhanced automated threat detection for firewalls.	Network Security
[20]	Machine Learning	AD-based IDS using ML techniques focusing on Isolation Forest and SVM.	Isolation Forest Model, SVM	Precise and robust AD methods for secure cloud infrastructure.	Cybersecurity, Network Protection
[21]	Machine Learning	Detect DDoS attacks by analyzing network traffic.	NB, DT, SVM, RF	Effective DDoS attack detection systems.	Cybersecurity
[22]	Recurrent Neural Networks (RNNs)	Exploring various RNN architectures for predicting characteristics of future network traffic based on past traffic.	RNN, LSTM, GRU	Volume prediction and novel areas of protocol and distribution prediction.	Network Traffic Analysis, Cyber Security
[23]	Hybrid DL (RNN, LSTM)	RNN with LSTM is used to detect malware threats in IoT networks.	RNN with LSTM	Identifying potential threats in IoT networks with high accuracy.	IoT and Network Security
[24]	Recurrent Neural Networks (RNNs)	Network traffic prediction, including volume, protocol, & distribution.	RNN, LSTM, GRU	Outperforming standard statistical forecasting models.	Network Traffic Analysis, Cybersecurity
[25]	ML for Anomaly-Based IDS	Investigating anomaly-based IDS using ML techniques.	Isolation Forest Model, SVM	Develop a precise and robust IDS for complex data environments.	Cybersecurity, Network Protection
[26]	Regression Analysis for IDS	Dimensionality reduction and prediction in NIDS.	Regression Methods	Enhance accuracy and efficiency in network intrusion detection.	Network Security
[27]	ML for Network Threat Detection	An AI-powered network threat detection system employs ML algorithms.	LightGBM, RF, KNN	Achieve high accuracy in network threat identification.	Network Security
[28]	Distributed DL for IDS	Implementing a distributed DL approach for IDS in industrial control systems.	DL, Big Data Techniques	Enhance intrusion detection in industrial environments.	Industrial Cybersecurity
[29]	Multi-class SVM and CHAID DT	Predicting next-generation cyberattacks in IoT systems with optimized methods	Multiclass SVM, CHAID DT	Enhance cyberattack prediction accuracy in IoT.	IoT Security
[30]	Real-time Network-Based AD	Real-time AD & malware analysis for SMEs & smart homes.	Real-time Detection	Provide security solutions for SMEs and smart homes.	Cybersecurity
[31]	AI and SVM-Based DL	Using AI and SVM-based DL for network intrusion detection.	SVM-Based DL	Advance network security with AI-driven intrusion detection.	Network Security
[32]	HSDL-Based Threat Detection Framework	Intelligent threat detection for IoT networks using an HSDL framework.	HSDL Framework	Enhance threat detection in IoT networks.	IoT Network Security
[33]	Advanced Port Scanning Techniques	Improving network visibility and security through advanced port scanning.	Advanced Port Scanning	Strengthening network security and visibility.	Network Security
[34]	IDS and Network Cyber-attack Simulation	OD1NF1ST system designed for IDS & avionics network cyberattack simulation.	OD1NF1ST System	Simulate and detect intrusions in avionics networks.	Cybersecurity
[35]	TOR Artifacts and Traffic Analysis	Analyzing TOR artifacts and traffic in Windows 11 for cybersecurity research.	Virtual Lab and Dataset Creation	Realizing the TOR network usage and implications in Windows 11.	Cybersecurity
[36]	Intelligent URL Classification System	Developing a system to identify and classify malicious URLs.	URL Classification	Classify and identify malicious URLs effectively.	Cybersecurity

III. RESEARCH GAPS

The comprehensive literature review has highlighted several significant trends, techniques, & advancements in network security. While many studies have greatly contributed to our knowledge of cyber threats & how to address them, our research aims to address important knowledge gaps identified in these studies.

Comprehensive combination of hybrid learning models:

Several research studies focused on particular areas, including intrusion prevention and AD. There must be a discernible gap in attaining full integration, even with the effectiveness of hybrid models that combine DL and conventional ML techniques. Our research seeks to close this gap by offering a complete solution combining several machine-learning methods into a coherent framework that can handle various network security issues.

Comprehensive data source utilization for IoT security:

Though the SoTA on IoT security-based ML is expanding, a significant gap exists in using various data sources. We try to close this gap by developing massive datasets, applying sophisticated preprocessing methods, and guaranteeing a comprehensive analysis of IoT networks' rich and varied data landscape.

Synergistic integration of usual & modern learning methods:

Studies that preexist frequently concentrate on either SoTA DL or ML methods for IDS. Combining these techniques requires more understanding to produce a more complex and unique detection system. To close this gap, our project smoothly merges classic and modern learning techniques, offering a complete defense against known and unknown threats.

Universal adaptability of deep learning approaches:

While some studies currently exhibit the efficacy of DL models in particular contexts, more research is required to guarantee their general adaptability across multiple network environments. By expanding the use of DL models beyond particular use cases, our research expands on previous work by making them broadly usable and flexible enough to be used in various contexts, ranging from public infrastructure systems to corporate networks.

Optimization of ML algorithms for real-time response:

The modern studies have accredited the significance of using optimized ML models; however, there is still a lack of grasping of how they simultaneously optimize for improved detection rates and integrate real-time data processing capabilities. Our work prioritizes optimization and real-time reactivity, thus ensuring the system is accurate and flexible when addressing possible threats.

Holistic datasets utilization for enhanced network security:

Previous research frequently concentrates on specific aspects of creating datasets for network security, including information unique to the IoT or models based on graph principles. By combining different data types, such as regular network information, special IoT details, and artificial data for training, our work seeks to close this gap by using graph-based models that enhance the system's understanding of complex network traffic patterns and connections.

Dynamic and Responsive Systems for Network Security:

Although some researchers examine the need for dynamic and responsive systems in network security, a gap exists in assuring the compatibility of optimum ML algorithms with real-time data analysis. To enable the system to respond quickly to rapidly advancing network conditions and developing threats, our project excels at optimizing algorithms for efficiency and ensuring their compliance with real-time data analysis.

Proactive Network Security with Predictive Capabilities:

The potential of AI-powered network threat detection and predictive security measures is covered in the literature under proactive network security with predictive capabilities. However, more research is necessary to fully utilize ML algorithms that can learn from immense datasets to anticipate and mitigate risks. By emphasizing predictive skills and effective data processing, our research closes this gap and establishes our project as a pioneer in proactive network security.

Improving current security measures via advanced AD methods:

Although several studies examine the integration of ML with firewalls and sophisticated AD algorithms, further research is necessary to guarantee that the system is not merely an add-on but a major improvement to current security measures. To provide a more comprehensive and reliable detection mechanism, this research builds technologies that will smoothly connect with current firewall infrastructures and improve their capabilities using robust AD algorithms.

Anticipation of traffic patterns & threats in network traffic:

Although some research has focused on analyzing network traffic for DDoS attack detection and using recurrent neural networks (RNNs) for traffic prediction, a clear understanding of the best methods to integrate advanced data analysis techniques and enhance the applicability of RNNs is still lacking. By utilizing cutting-edge data analysis methods and enhancing the application of RNNs, our technology closes this gap by facilitating proactive traffic pattern prediction and efficient DDoS attack identification and mitigation.

Continuous advancements & realistic training in cybersecurity:

Previous research emphasizes the generation of artificial network traffic datasets and investigating innovative ML methods for current network-based attacks. More is necessary to guarantee ongoing innovation and practical cybersecurity training. We embrace this need by integrating unique and proven machine-learning approaches and combining authentic and synthetic data to ensure the system has the latest tools and strategies to battle emerging threats efficiently. Flexible Response to IoT-Specific Dangers and Wider Network Issues: More research needs to be done to achieve better adaptability to IoT-specific threats and wider network challenges, even though some studies are more concentrated on specialized models for IoT and networking malware detection. Our research adopts a comprehensive strategy and flexible solution combining sophisticated ML techniques with regression analysis to address general network challenges and risks unique to the IoT.

Advanced analytics & detection of abnormal network packets:

Several research studies emphasize the identification of abnormal network packets using advanced analytics, highlighting the necessity for accurate and sophisticated detection methods. Our effort advances in this field by integrating adaptive algorithms with real-time analysis, improving the system's ability to recognize and react quickly to abnormal packets. This is an important part of preventing future breaches and preserving network integrity.

Scalable solutions via distributed deep learning techniques:

While some research explores distributed DL techniques for network data analysis, we still need to learn more about simplifying architecture without sacrificing manageability and efficiency. Our research uses distributed DL techniques while upholding a balanced design to guarantee scalability and applicability in various scenarios. Next-Generation Cyber Attack Prevention Using Prediction Strategies: Previous research highlighted the importance of network security foresight by discussing prediction strategies for potential cyberattacks in the future. Our research, which utilizes ML algorithms capable of processing large datasets and enhancing predictive skills, aligns closely with the emphasis on next-generation cyberattack prevention.

Real-Time network AD: Considering speed & efficiency:

Although some research has focused on real-time AD in network traffic, it is still necessary to ensure that data density or complexity does not jeopardize it. Our project is designed to be as fast and efficient as possible, with real-time AD being an essential component of how well it works to identify and neutralize possible threats quickly.

IV. REQUIREMENTS SPECIFICATION

A. Stakeholders

Table 2 summarizes the system stakeholders, their interaction with the system, and the importance of the role. Eleven key groups involved in network security are categorized, outlining their roles and contributions. It highlights the diverse functions, from implementing protocols to regulatory compliance, providing a brief overview of each stakeholder's impact on the security infrastructure.

B. Platform Requirements

The system is client- and server-based. Standard web browsers on the client side provide easy system interaction. Operating system updates boost performance. The server-side requirements use Ubuntu Linux and Windows Server 2016, enhancing system functionality. The firewall and router platform handles LAN activity using pfSense, connecting all LAN devices to the internet. CPU, RAM, and storage must support the system load.

Table 2. Stakeholders and their interactions and roles

Stakeholder	Interaction with System	Importance of Role
Network Security Professional	Security protocol implementation, threat detection, dedicated tools, and countering security incidents.	Vital role in ensuring the efficiency of the implemented security measures, as the above stakeholder contributes to day-to-day security operations.
Organizations and Enterprises	Implementing network security and running the integration with the current infrastructure.	Adoption of the system and alignment with the overall organizational objectives and security requirements.
IT Departments	System integration with the daily network operations and maintaining the network functions.	The system's performance has an impact, as the influence reaches the workload and efficiency in network security management.
System and Network Admins	System configuration & implementation to ensure compatible network components.	An essential contribution to the system deployment with a direct impact on the adaptability and integration.
Security Software Developers	Elevating system deliverables for future updates and feature enhancement.	Developing security software based on system features while keeping up to date with industry trends and new technology.
Regulatory Bodies and Compliance Authorities	System testing to ensure compliance with regulatory requirements & standards.	Potential contribution to developing and adapting the cybersecurity regulations and compliance requirements.
Educational Institutions	System integration is delivered with academic curricula.	Impact the academic advancement of cybersecurity knowledge.
End Users & General Public	Enhance & strengthen system security measures.	System success benefits with an impact leading to a safer online environment.
Government Agencies	System utilization for national cybersecurity strategies.	Contribution to the national critical infrastructure and network strategies.
Cybersecurity Service Providers	System enhancements & integrated system features.	Developing commercial cybersecurity solutions based on system outcomes.
Investors and Funding Organizations	Oversee the project's progress and its deliverables for future investments.	The goal is to attract investments and enable ongoing development and implementation.

In light of IDS and network analysis, Snort has played an essential role since it originated in the late 90s. Cisco now employs it as a service or feature on pfSense to oversee incoming and outgoing traffic [37][38]. In addition to *Suricata*, another IDS tool performs tasks similar to Snort, such as logging activities (TLS/SSL, HTTP, and DNS queries) [39]. Both Snort and Suricata are configured on the pfSense virtual router to enable real-time traffic analysis and potential threat detection [37][38][39]. *Wireshark* is also a powerful network protocol analyzer that provides detailed insights into network activities [40]. The tool provides initial details for capturing and analyzing network packets, offering a comprehensive range of features that support the machine-learning model and enhance system accuracy. Additional software tools like Kali Linux are incorporated into the lab's environment for ethical hacking and penetration testing platforms. *OWASP Broken Web App* (BWA) is the primary testing target, as it simulates various network weaknesses and vulnerabilities. Proper configuration of virtual lab network settings is necessary to ensure compatibility and efficient communication. Compatible plugins for Snort and Suricata are enabled on pfSense, extending the functionality of the IDS systems and enhancing the lab's security. The research project also includes monitored attack simulations to evaluate network security. The simulations involve probing the server, port scanning, and vulnerability scanning with tools like Nikto, Wapiti, Skipfish, Wpscan, and FUZZ testing targeting URLs through the fog tool. The simulations are crucial in identifying security vulnerabilities within the target system to assess the overall network security structure [42].

C. Function/Non-Functional Requirements

Table 3 shows seven necessary network security components, functionalities, and constraints. It offers a concise, inclusive view of system requirements, deepening elements like user access and browser compatibility. Table 4 shows nine key non-functional requirements, including performance & security measures for network security, offering examples and standards for system durability and effectiveness.

D. Other Requirements

API Restrictions: The system must align with security guidelines and only use authorized APIs for data exchange. Any external API should follow industrial guidelines & best practices to provide authentication and smooth data transmission [43].

Data Transmission: The system mandatorily prioritizes using secure data transmission protocols like HTTPS, allowing communication between multiple components. Additionally, any protocol used for data exchange must be cited along with its implementation through predefined standards.

Data Storage Formats: Any data stored inside the system (including logs & configuration files) is required to follow standardized guidelines, providing ease of fetching, retrieval, and analysis. Data storage formats must be aligned & compatible with industry-standard data storing, processing, & reporting tools.

Compliance with Regulations: The system should be fully compliant, avoiding failures in following up with ongoing updates in worldwide regulations.

Backup and Recovery: The system has to put in force ordinary backup procedures to prevent information loss. A robust recovery plan in the area must be in place, specifying techniques for restoring devices to a useful state after a disruption.

User Authentication Standards: User authentication methods have to follow industrial requirements. Multi-factor authentication is one of the best practices for achieving personal account security.

Logging and Auditing: To assist further risk detection, comprehensive and real-time logging will be implemented on multiple interfaces, including logins and adjustments. Audit path logs are to be maintained for analytic and forensic purposes.

System Performance Monitoring: Continuous tracking tools are utilized to evaluate the system's performance in real-time, and overall performance thresholds & signals must be configured to notify administrators of any system failures.

Notification Mechanisms: The system should offer configurable notification options to alert administrators to critical activities or protection incidents, ensuring timely notifications that include relevant records for an effective incident response.

Usability Guidelines: The user interface must comply with usability suggestions to ensure a wonderful and smooth user experience for administrators and users interacting with the system. Additionally, ordinary usability checking must be conducted to detect and mitigate user-interface problems.

Table 3. Functional Requirements

Requirement	Input	Output	Processes	Constraints	Example
User Access	Web Browser	User Interface	Interaction with Web Interface	Essential	Valid User & Password Login
Browser Compatibility	Compatible Browsers	Compatibility Status	Checks & Updates	Recommended	Latest Version of Browsers
OS Support	OS Specifications	Server-side Functionality	Installation & Execution on OS	Essential	Server: Ubuntu & WS2016
Firewall & Router	PfSense Configuration	Network Management	Configuration for network control	Essential	Blocking Unauthorized Traffic
Real-time Analysis	Snort/Suricata Config.	Network Activities Detection	Installation & configuration of IDSs	Essential	Log threats in milliseconds
cPlugin Configuration	Plugin Configurations	Enhanced IDS Functionality	Configuring Compatible Plugins	Essential	Identify Network Activities
Network Settings	Network Configuration Parameters	Efficient Flow of Network Traffic	Adjustments for Seamless Communication	Essential	Allowing Seamless Comm.
Attack Simulations	Various Attack Simulation Parameters	Identification of Security Vulnerabilities	Probing, port scanning, fuzz testing, etc.	Recommended	Sending Packets to Target for Testing Reachability.

Table 4. Non-Functional Requirements

Non-Functional Requirement	Example
Performance	The system should respond to user actions within 2 seconds in normal operating circumstances.
Storage Limits	The system should be able to store network traffic logs for at least 6 months.
Code Quality	The source code must align with the industry-standard coding standards and is subject to regular code reviews.
Documentation	Comprehensive documentation, which includes installation manuals, is provided.
Accessibility	The interface must align with specific standards to ensure accessibility for users with special needs.
Security	The system encrypts sensitive data, including user credentials and traffic logs.
Reliability	The system must have a minimum uptime of no less than 99% to ensure continuous monitoring, threat detection, and user accessibility.
Scalability	The system is required to survive additional traffic by at least 20% without showing any performance degradation.
User Interface Ease-of-Use	The interface adheres to best design practices to guarantee administrators' ease of use.

V. Proposed Architectural Design

A. Overall System Architecture

Fig. 1 presents the overall architecture for the proposed threat-hunting model. It's composed of several elements stated earlier, including the Kali Linux attacker, the WWW module, the PFSense router, the firewalls, the OWASP BWA Target, Ubuntu monitors, the switch LAN, and the DMZ area with Windows Server 2016.

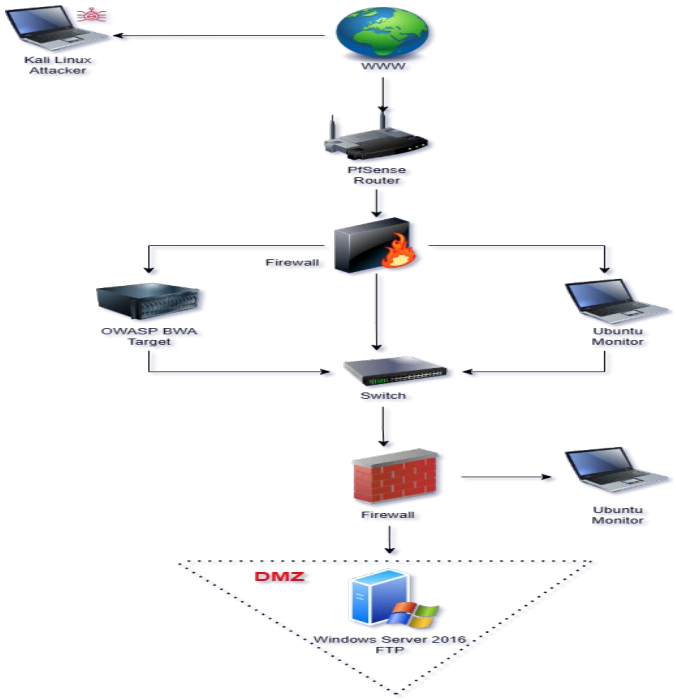


Figure 1. Architecture of System

As far as we have developed our threat model and generated our comprehensive datasets, in this phase, we focus on setting up the data processing pipeline and model development to address the project's goals. Our objective is to develop a robust model that distinguishes between DoS attacks and normal traffic, which will later be extended to include multiple types of network attacks. To begin with, the goal of building a single model that will only differentiate DoS traffic from normal traffic is to validate the models we will be testing and deploying for all of the different types of attacks. This is also done to ensure that we are processing the data correctly and then choosing the suitable model that works best for our mixture of data. We benefit from this in multiple ways, including 'getting to know' our data. This section will detail the overall strategy for the data preprocessing stage, which includes preparing the data, selecting features, and choosing models to distinguish between normal network traffic and various attack vectors. Figure 2 describes the leading architecture of the system, detailing the process from data acquisition through all the data preprocessing & other phases to the model deployment phase.

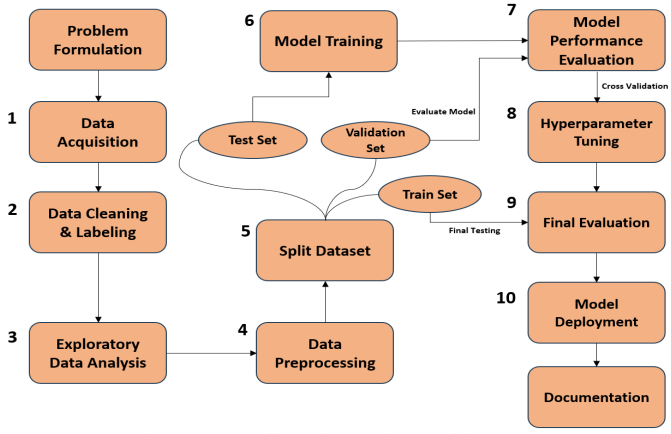


Figure 2. Overall Implementation Architecture

B. Dataset Acquisition/Creation

This section provides a detailed description of the procedures involved in framing a comprehensive dataset specifically designed for activities related to network traffic detection. The preparation of this dataset is extremely important since it boosts

the application and testing of precise machine-learning models. The creation process is efficiently achieved through the steps:

- **Merging of Network Traffic Data:** Initially, data was sourced from various repositories, including Snort, Suricata IDS, and Wireshark, and it was merged carefully. These special datasets are blended by aligning them with an integrated time reference. Very lengthy efforts are made to detect and remove duplicate entries, ensuring maximum data accuracy.
- **Conversion to a Common Format:** After the merging phase, the dataset must be converted into a universally recognized and standardized format. This transformation requires converting data structures from IDS records and Wireshark PCAP files into a uniform and recognized format. Essential features such as source and destination IP addresses, ports, protocols, packet sizes, and timestamps are explicitly defined. These features are then unified, resulting in a unified data set. The data frame is then converted into a CSV file, making data sharing and additional analysis easier [44].
- **Data Improvement and Annotation:** Data improvement is the final & vital step in dataset preparation. Additional information is carefully introduced to enhance the analysis of network traffic. Manual annotation categorizes. Each record is labeled "benign" or "suspicious" with extreme caution. This accurate method helps develop an inclusive, detailed dataset ready for in-depth analysis & research within the context of network traffic security.

Our dataset has over 6 million traffic samples generated by attacking the environment we created. This approach grants us the privilege of tuning the dataset, controlling the attacks, and controlling the samples and the dataset's features. Table 5 shows the dataset's traffic types, numbers, and details.

Table 5. Normal and Abnormal Traffic Details

	Normal Traffic	Abnormal Traffic
Number	Over 3,000,000 Samples	3,724,869 Sample
Source	Normal traffic is generated by using multiple techniques	Gathered from multiple types of attacks (15 attacks)

The pie chart in Figure 3 presents how the samples have been distributed. Creating and generating the dataset in our lab allowed us to manipulate and adjust the data to fit our model. Our dataset can be formed in various forms, including binary datasets for classifying normal and abnormal traffic, a multi-class dataset featuring samples of each attack, and a dataset containing normal traffic samples alongside labeled abnormal traffic samples for each attack.

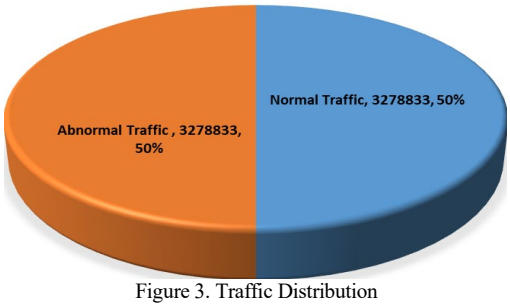


Figure 3. Traffic Distribution

The table below, Table 6, displays the attacks we simulated and the number of samples we generated for each. Note that we are currently considering and enhancing the dataset. We aim to improve the data and make the number of samples for each attack equal or semi-equal to each other.

Table 6. Number of Samples for Each Attack

Attack Name	Number of Samples
MITM-ARP-Spoofing	96926
ssh -brute force	139761
FTP—brute force	121917
DDOS-ICMP	222254
DDOS-raw	195148
DDOS-UDP	318210
DOS	593624
Exploiting-FTP	13573
Fuzzing	117559
ICMP Flood	240083
SYN Flood	739157
Port Scanning	162320
Remote Code Execution	278874
SQL Injection	270000
XSS	215463
Total	**3724869**

The traffic distribution proportions for the normal vs. anomaly (attacks) are displayed in Figure 4. The illustrated pie chart shows the balanced dataset between normal and anomalies.

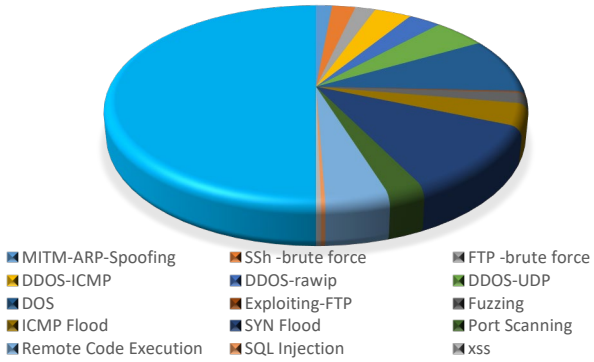


Figure 4. Normal vs. abnormal Traffic Distribution

This dataset has over 60 different features (provided in Table 6 below). This high number of features collected for each attack will assist in creating a highly precise and accurate model that can classify any traffic fed to it. The following section will name some of the more noteworthy features and provide a brief description of each of them. In addition, these features have the highest weight in determining what the incoming traffic represents to ensure it is classified precisely, achieving the least number of false positives and negatives possible to fit your needs.

Table 1. The Dataset Features

No	Feature	Description
1	Time	The time at which the packet was captured.
2	Source	The packet contains the source IP address.
3	Destination	This is the destination IP address of the packet.
4	Protocol	The network protocol used is TCP, UDP, or ICMP.
5	Length	The packet length in bytes.
6	Info	A packet description of the communication nature.
7	Frame Number	A frame unique identifier.
8	Frame Length	The length of the whole frame
9	Same Time	What time was the frame captured?
10	Frame Time (Epoch)	The timestamp of the frame in epoch format.
11	Frame Protocols	Protocols used in a data frame in a hierarchy format.
12	Ethernet Source	The MAC address of the source device is located in the Ethernet frame.

13	Ethernet Destination	The MAC address of the destination node is located in the Ethernet frame.
14	Ethernet Type	The Ethernet frame type indicates the payload protocol (e.g., IPv4, IPv6).
15	IP Source	The source IP address in the IP packet is where the packet came from.
16	IP Destination	The IP address destination in the IP packet.
17	IP Length	The IP packet length, including the header and payload.
18	IP TTL	Time to Live (TTL) is an IP header field representing the number of hops before the packet is discarded.
19	IP Flags	The 3-bit flag in the IP header, such as Don't Fragment (DF) and More Fragments (MF).
20	IP Fragment Offset	The offset of the fragment in the original IP packet.
21	IP Protocol	Network layer protocol: TCP, UDP, or ICMP.
22	IP Version	The protocol of the Internet Protocol IPv4 or IPv6
23	IP DSCP Field	The defined Services Code Point (DSCP) field indicates the packet's priority in the IP header.
24	IP Checksum	The checksum value for the IP header.
25	TCP Source Port	The source port number is in the TCP header.
26	TCP Destination Port	The destination port number is in the TCP header.
27	TCP Length	The length of the TCP packet, including the header and payload.
28	TCP Sequence Number	The sequence number in the TCP header is used to reorder and reassemble packets.
29	TCP Ack. Number	The acknowledgment number in the TCP header acknowledges receipt of data.
30	TCP Flags	The TCP header flags indicate control information such as SYN, ACK, FIN, etc.
31	TCP SYN Flag	Synchronization request flag in the TCP header.
32	TCP ACK Flag	The acknowledgment flag in the TCP header.
33	TCP FIN Flag	Indicates the finish flag in the TCP header.
34	TCP RST Flag	Indicates the reset flag in the TCP header.
35	TCP Window Size	Receive window size in the TCP header specifies the data size to be sent before an ACK is received.
36	TCP Checksum	The checksum value for the TCP header.
37	TCP Stream	The TCP stream identifier.
38	UDP Source Port	The source port number is in the UDP header.
39	UDP Destination Port	The destination port number is in the UDP header.
40	UDP Length	The length of the UDP packet, including the header and payload.
41	UDP Checksum	The checksum value for the UDP header.
42	ICMP Type	The type of ICMP message, if the packet is ICMP.
43	ICMP Checksum	The checksum value for the ICMP header.
44	HTTP Request Method	The HTTP request method, such as GET or POST.
45	HTTP Request URI	The Uniform Resource Identifier (URI) is requested in the HTTP request.
46	HTTP Request Version	The version of the HTTP protocol used in the request.
47	HTTP Full URI	The full URI, including the request method, URI, and version.
48	HTTP Response Code	The HTTP status code is returned in the response.
49	HTTP User-Agent	The user-agent string indicates the client software.
50	HTTP Content-Length	The length of the HTTP content in the payload.
51	HTTP Content-Type	The type of content in the HTTP payload (e.g., text/html, application/json).
52	HTTP Cookie	The value of the Cookie header in the HTTP request.
53	HTTP Host	The value of the Host header in the HTTP request.
54	HTTP Referer	The value of the Referer header in the HTTP request indicates the referring page.
56	HTTP Location	The value of the Location header in the HTTP response indicates the URL to which to redirect.
57	HTTP Authorization	The value of the Authorization header in the HTTP request indicates credentials for authentication.
58	HTTP Connection	The value of Connection header in an HTTP request indicates if the connection should be kept alive.
59	DNS Query Name	The domain name is being queried in a DNS request.
60	DNS Query Type	The type of DNS query, such as A, AAAA, MX, etc.

C. Data Preprocessing

Data preprocessing plays a pivotal role in ensuring the quality and consistency of inputs for ML models. This section describes the methodology adopted to prepare the dataset. This includes data cleaning, transformation, & partitioning. The preprocessing workflow comprises three key phases: (1) library configuration, (2) structured data transformation, and (3) stratified data partitioning. Fig.5 shows a high-level overview of the preprocessing pipeline.

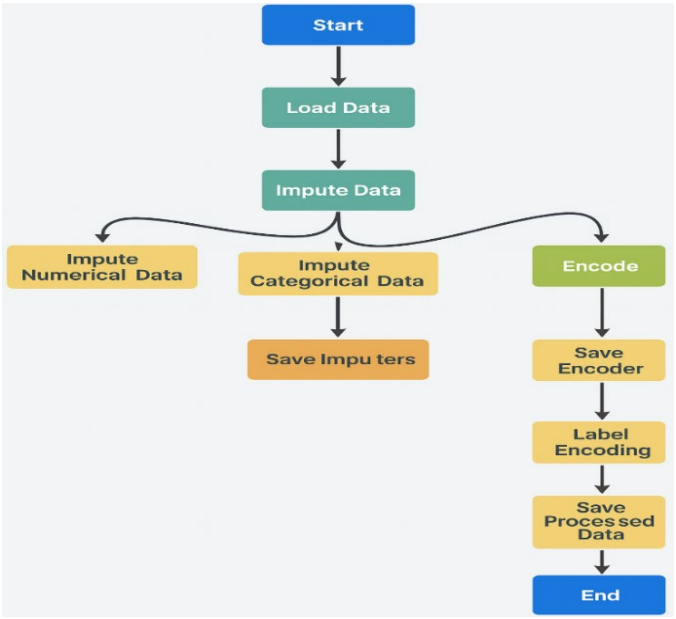


Figure 5: Data Preprocessing Functions Flowchart

Library Setup: A comprehensive set of Python libraries was employed to facilitate data manipulation, preprocessing, model development, and evaluation. Core libraries included pandas, NumPy, and matplotlib for data handling and visualization, while ML operations leveraged scikit-learn, imblearn, and torch. Serialization of fitted preprocessing objects was handled using the pickle module to ensure consistency across model training and deployment phases.

Data Transformation Pipeline: A dedicated preprocess_data function was implemented to clean and transform raw datasets. The process comprised four sequential stages:

- **Imputation**: Missing values were addressed using the SimpleImputer. Numerical columns were filled with the meaning, while categorical columns used the most frequent category. The imputers were serialized as numerical_imputer.pkl and categorical_imputer.pkl.
- **Encoding**: This includes (a) Ordinal Encoding: Categorical features were transformed into numerical representations using OrdinalEncoder, and (b) Label Encoding: The target label was encoded using LabelEncoder. Both encoders were saved to disk for reuse during inference.
- **Feature Scaling**: Continuous features were standardized using StandardScaler, normalizing their distribution by removing the mean and scaling to unit variance. The scaler was stored as scaler.pkl.
- **Persistence**: The transformed dataset was saved, and all preprocessing components were serialized to ensure reproducibility.

A step-by-step breakdown of the preprocessing function is provided in Table 13.

Table 13: Breakdown of Data Preprocessing Steps in the preprocess_data Function

Step	Description
Imputation	This step handles missing data in the DataFrame:
Imputer	SimpleImputer: mean value to fill in missing numerical values.
Categorical Imputer	SimpleImputer uses the most_frequent strategy to fill in missing values in categorical columns.
Pickling	Fitted imputers to numerical_imputer.pkl & categorical to_imputer.pkl.
Encoding	This step converts categorical variables to numerical values:
Ordinal Encoder	It uses an ordinal encoder to transform categorical features into numerical values.
Pickling	Saves fitted encoder to the encoder. Pl using the pickle module.
Encoding	This step encodes the target label column into numerical values.
Label Encoder	LabelEncoder is used to transform the target label into numerical values.
Pickling	Saves fitted label encoder to label_encoder.pkl using pickle module.
Scaling	This step standardizes numerical features:
Standard Scaler	It uses StandardScaler to standardize numerical features by removing the mean and scaling to unit variance.
Pickling	Saves the fitted scaler to the scaler. Please use the pickle module.
Integration	The processed data frame undergoes several transformations in sequence: imputation, encoding, label encoding, and scaling.

Stratified Data Splitting: The preprocessed data was partitioned using stratified sampling via StratifiedShuffleSplit to ensure balanced class distribution across datasets. Initially, the dataset was split into 70% training/validation and 30% testing. Subsequently, the training/validation set was split into 90% training and 10% validation subsets. This preserved class proportions across all partitions, essential for mitigating data imbalance and enhancing model generalizability. Each subset was exported to CSV files (X_train, y_train, X_val, y_val, X_test, y_test) for reproducibility and consistent benchmarking. A functional overview of the stratified splitting procedure is outlined in Table 14.

Table 14: Breakdown of the StratifiedSplit Function

Drop Label Column	Separates features (X) and the target label (y) by dropping the label column from the DataFrame.
Initialize Splitter	Initializes StratifiedShuffleSplit with one split, 30% test size, and a random state for reproducibility.
First Split	Splits the data into training/validation and testing sets while maintaining class proportions.
Split Indices	Obtains the indices for the train/validate/test sets.
Create Train/ Test Sets	Creates the X_train_val, X_test, y_train_val, and y_test sets using the obtained indices.
Second Split	Initializes another StratifiedShuffleSplit with one split, a 10% validation size, & random state for reproducibility.
Validation Split	Split the training/validation set into separate training and validation sets while maintaining class proportions.
Save to CSV	Saves the X_train, y_train, X_val, y_val, X_test, and y_test sets to CSV files for future use.
Return Sets	Returns the training, validation, and testing sets (X_train, y_train, X_val, y_val, X_test, y_test).

VI. Model Implementation

ML and DL models were implemented, trained, and evaluated using consistent preprocessing and validation procedures to develop a robust classification framework. This section outlines the training pipeline, model architectures, and performance evaluation strategies.

A. ML Models

Four classical ML algorithms were employed: LR, DT, RF, and NB. Each model was implemented with hyperparameter tuning and evaluated using 10-fold cross-validation to ensure generalizability and reduce overfitting. The overall workflow for ML models is illustrated in Figure 6. For all ML models:

— Cross-validation was stratified with fixed seeds for reproducibility.
— Accuracy scores were computed for training/validation/test splits.
— Trained models were serialized using the pickle library.

Model configurations included:

— **LR model**: Regularized linear model, suitable for binary classification.
— **DT model:** Tuned with constraints on tree depth, minimum samples per leaf, and splitting criteria.
— **RF model**: An ensemble of DTs, optimized for feature randomness and tree count.
— **NB model: A** probabilistic model assuming conditional independence between features.

Performance metrics and cross-validation averages were recorded across folds. Refer to Table 15 for detailed model training steps.

Table 15: Breakdown of the detailed model training steps

Step	Description
Initialize Model	Initializes the "LR, DT, RF, or NB" model with specified parameters.
K-Fold Validation	10-fold cross-validation, shuffling, & a fixed random state for reproducibility.
Training Loop	Training on different folds, validating & testing sets, & storing accuracy scores.
Model Save	Saves the trained model to a file using Pickle.
Performance Plot	Plotting model performance of different folds for training, validation & test sets.
Print Averages	Prints the average training, validation, and test accuracy scores across all folds.

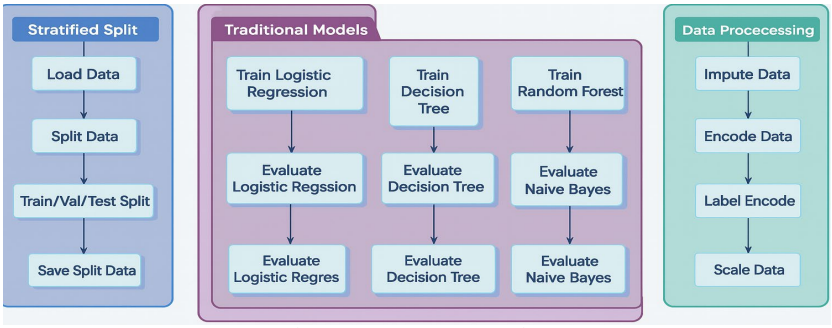


Figure 6: ML Process Overview

B. DL Models

To capture more complex patterns, several DL architectures were constructed using PyTorch. These include FNN, MLP, SCNN, and DCNN. All models were trained using GPU acceleration (when available). Training loops recorded per-epoch losses and accuracies on training, validation, and test datasets. The model parameters were saved and evaluated using accuracy, precision, recall, F1-score, and confusion matrix analysis. The overall workflow for DL models is illustrated in Fig.7. Table 16 also provides a breakdown of each DL model and training procedure.

Table 16. Breakdown of the DL Models

Model	Description
FNN	A feedforward neural network with three fully connected layers, ReLU activations, and dropout for regularization.
MLP	A multi-layer perceptron with dropout, batch normalization, and two fully connected layers.
SCNN	A shallow convolutional neural network with one convolutional layer, batch normalization, ReLU activation, and average pooling.
DCNN	A DCNN with two convolutional layers, batch normalization, ReLU activations, and average pooling.

C. Training Pipeline Integration

The core training function devised the intact workflow, including data preprocessing, stratified splitting, model training, and evaluation and visualization: output of performance metrics and training curves. The function ensures reproducibility, maintains modularity, and supports evaluation across diverse model types.

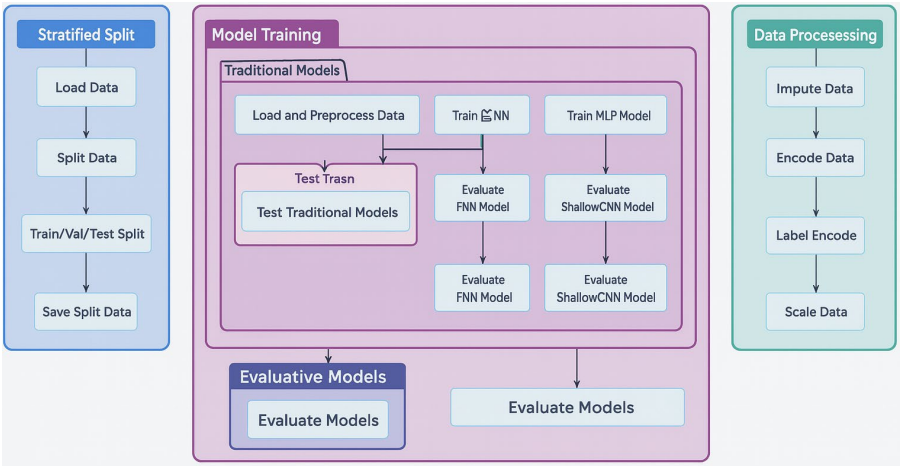


Figure 7: DL Process Overview

D. Handling Data Imbalance

To mitigate class imbalance, Synthetic Minority Over-sampling Technique (SMOTE) was applied. SMOTE generates synthetic samples by interpolating between minority class instances and their k-nearest neighbors. This strategy notably improved model performance on minority classes. This step improves the model's fairness and recall, especially for underrepresented classes. The synthetic data creation process involves:

— Identifying k-nearest neighbors for each minority sample.
— Generating new samples using linear interpolation.
— Reconstructing the dataset with original & synthetic samples.

E. Deployment and GUI Integration

A lightweight graphical user interface (GUI) was developed using **Streamlit** to facilitate real-time interaction with the trained models. Users can:

— Load network traffic data (.csv or Wireshark).
— Start monitoring live data streams.
— View per-packet predictions in real time.
— Save the results for offline analysis.

The GUI integrates trained models via serialized .pkl or PyTorch model files and delivers a user-friendly environment for non-technical users.

VII. Model Evaluation and Testing

A. Evaluation Strategy

Robust evaluation of ML and DL models is crucial for verifying generalization performance. We employed binary, multiclass, and anomaly-based classification tests using various performance metrics to assess each model's effectiveness in intrusion detection. Testing covered a wide range of attacks with detailed per-model metrics, including accuracy (ACC), precision (PRC), recall (RCL), F1-score (FSC), specificity (SPC), area under the ROC curve (AUC), and detection time (Ts).

B. Performance Metrics

The following standard metrics were used:

- **Accuracy (ACC):** Ratio of correctly classified to the total number.
- **Precision (PRC):** Ratio of true positives among predicted positives.
- **Recall (RCL)**: Ratio of true positives among actual positives.
- **F1-Score (FSC):** Harmonic mean of precision and recall.
- **Specificity (SPC):** Ratio of true negatives among actual negatives.
- **Area under curve (AUC)**: indicates model discrimination power.
- **Detection Time (Ts):** time to classify a fixed number of instances (N).

C. Binary Classification Results

Fifteen binary classifiers were trained, each targeting one attack type against normal traffic. DL models—especially DCNN and SCNN—consistently outperformed traditional classifiers regarding detection accuracy and generalization across most attack types. Notably:

— **DCNN** achieved near-perfect accuracy (≥99.9%) for DoS, DDoS-ICMP, and fuzzing attacks.
— **RF** and **SCNN** offered a strong balance between performance and speed for most attacks.
— **NB** and **DT** underperformed on complex attack patterns, demonstrating poor generalization.

Table 17 summarizes the top-performing models for each attack type, highlighting key evaluation metrics such as ACC, PRC, RCL, AUC, and Ts. The table identifies the most effective classifiers and their capability to accurately detect various network attacks from normal traffic. This comparative overview assists in selecting the optimal model based on performance and computational efficiency for each specific attack scenario.

D. Anomaly-Based Detection

A binary classifier was built to distinguish anomalous traffic (all attack types combined) from normal behaviour using 1,623,413 training samples and 851,517 test samples. The **RF, FNN,**

SCNN, and **DCNN** models reached perfect or near-perfect accuracy (≈1.000), with high ROC AUC and specificity. LR & MLP demonstrated balanced but slightly lower accuracy and faster prediction speed.

Table 17. A summary of high-performing results per attack

Attack Type	Top Model	ACC	PRC	RCL	AUC	Ts (s)
DoS	DT	0.9997	0.9995	1,000	1.000	0.015
DDoS-ICMP	RF/DCNN	1.000	1.000	1.000	1.000	0.194
DDoS-Raw	DCNN	0.9999	0.9999	0.9999	0.998	2.712
DoS-UDP	DCNN	1.000	1.000	1.000	1.000	2.065
XSS	DCNN	0.9975	0.9950	1.000	1.000	2.647
FTP-BruteForce	DT	0.9915	0.9909	0.9911	0.991	0.009
FTP-Exploitation	DCNN	0.9195	0.9288	0.9195	0.988	2.027
Fuzzing	DCNN	0.9994	0.9992	0.9995	0.952	1.709
ICMP-Flood	DCNN	1.000	1.000	1.000	0.988	2.139
MITM-ARP	MLP	0.9993	0.9991	0.9993	0.999	1.192
Port Scanning	DCNN	0.9955	0.9950	0.9959	0.962	1.866
RCE	DCNN	0.9919	0.9920	0.9919	0.988	2.091
SSH-BF	NB	0.9939	0.9928	0.9946	0.994	0.031
SYN-Flood	RF/DCNN	1.000	1.000	1.000	1.000	2.070

E. Multi-Class Classification

A multi-class classifier was developed & evaluated on 851,517 samples representing 16 classes to evaluate the model's ability to classify attack types simultaneously. The results highlight:

- **DCNN** & **RF** achieved 99.9% accuracy and an F1-score.
- **DT** offered near-perfect accuracy with the lowest detection time (0.1478 s), ideal for real-time IDS.
- **MLP** & **NB** performed lower across most metrics.

Table 18 summarizes the highest-performing results for each ML model in the multi-class classification task. It includes key metrics such as accuracy, precision, recall, F1-score, AUC, specificity, and detection time. The table highlights the effectiveness of advanced models like DCNN.

RF and SCNN achieved near-perfect classification performance across all metrics. It also underscores the trade-offs between accuracy and computational efficiency, providing helpful details about model selection for multi-class IDSs.

Table 18. A summary of high-performing Multi-Class Classification results per ML model

Model	ACC	PRC	RCL	F1	AUC	IDSsTs (s)	SPC
DT	0.9999	0.9999	0.9999	0.9999	0.9999	0.1478	1.0000
RF	0.9999	0.9999	0.9999	0.9999	1.0000	3.7531	1.0000
DCNN	0.9999	0.9999	0.9999	0.9999	0.9995	13.9084	0.9999
FNN	0.9999	0.9998	0.9998	0.9998	0.9990	12.1578	0.9998
SCNN	0.9999	0.9998	0.9998	0.9998	0.9990	11.6989	0.9998
NB	0.9579	0.9639	0.9543	0.9571	0.9978	1.8786	1.0000
MLP	0.8923	0.9037	0.8849	0.8879	0.9100	10.9449	0.9037
LR	0.9591	0.9591	0.9591	0.9591	0.9800	0.5000	0.9591

F. Cross-Validation and Training Dynamics

Cross-validation plots revealed that:

- **DT and RF** exhibited signs of overfitting, with perfect training accuracy but fluctuating validation scores.
- **FNN, SCNN, and DCNN** showed consistent convergence trends, decreasing losses & increasing accuracy over epochs.
- **MLP** maintained steady accuracy but showed higher sensitivity to epoch changes and noise.

G. Summary and Insights

- **Best Overall Accuracy**: RF, DCNN, SCNN, and FNN.
- **Best real-time performance**: DT (low Ts, high ACC).
- **Trade-Offs**: DL models provide exceptional accuracy but have higher computational costs.
- **Recommendation**: RF or SCNN offers the best performance speed balance for high-throughput environments. For lightweight deployment, DT remains highly efficient.

VIII. Conclusion and Remarks

This work aimed to develop an inclusive threat-hunting model, which spans data generation to intelligent threat detection in communication networks. The holistic approach aims to construct a robust IDS capable of distinguishing normal from malicious traffic that classifies malicious traffic into 15 distinct attack types. The following key steps are to realize this ambitious goal:

- **Data Generation and Acquisition:** Over 6 million unique data samples have been generated by creating a virtual lab scenario with Red Team attack platforms and Blue Team defense platforms. This has brought the capability to capture a wide diversity of attack vectors and regular traffic, establishing a balanced and complete dataset for training and testing.
- **Data Preprocessing**: The well-established data preprocessing pipeline imputes, encodes, and scales each feature carefully. This ensures that the data fed to it is clean, balanced, and standardized, which is critical for IDS accuracy and reliability.
- **Model Implementation:** Implement ML and DL models and evaluate their performance well (ML models: LR, DT, RF, NB – DL models: FNN, MLP, SCNN, and DCNN).
- **Model Evaluation**: To evaluate the models, they are compared using the related metrics, such as accuracy, precision, recall, F1 score, ROC AUC, specificity, and

detection time. DT achieved the best performance in the Decision IDS models, with an accuracy level reaching 99.9%.

— **Testing and Validation:** The built models were tested meticulously using the most complex attack types, from which each performance was noted with no leniency in ruling out any predicted results. This testing has generated the reliability of the IDS under realistic situations.

— **Graphical User Interface**: A user interface was developed using the Streamlit library, making it easy for an end-user to interact with the IDS. These interfaces deliver real-time monitoring for traffic, classification results, and data input through CSV files or live packet-capturing tools like Wireshark.

IX. Future Directions and Recommendations

Having reached several milestones in the project, there remain a few avenues for future work that can take the IDS to an entirely new level of completeness:

— **Addition of attack types**: The 15 addressed within the project provide further depth to the comprehensiveness of the IDS. Using emerging attack vectors will keep the IDS up-to-date with the changing threat landscape.

— **Real-time Adaptation:** Real-time learning & adaptation mechanisms should be included to allow the IDS to evolve continuously as new threat patterns and data emerge by exploring techniques like online learning & adaptive methods.

— **Advanced Feature Engineering:** Features such as selection and extraction can improve the accuracy and efficiency of the model. These are techniques used for the preprocessing pipeline: These techniques include principal component analysis and automated tools for feature engineering.

— **Integration of IDS with Other Security Systems:** Such as firewalls & endpoint protection platforms, offers maximum security. Besides, interoperability with the SIEM system further enhances the threat correlation and incident response capability.

— **Scalability and Performance Optimization:** Contributing to the development of IDS scalability, it needs to be perfected in a way that it can operate in a networking environment that is highly scaled without performance compromise. To achieve this, we may consider techniques such as distributed computing, parallel processing, and cloud-based deployment.

— **Enhanced GUI Features:** With rich features, including detailed analytics dashboards and alert settings that can be customized per user requirements and other external data sources. This is the point at which interactive visualizations with full-fledged reports provide the necessary backup that helps perform an in-depth analysis of threats.

— **User Behavior Analysis:** Some methods of user behavior analysis that may be implemented in the current system will involve detecting anomalies according to their deviation from normal user behavior. This may be done using user profiling, AD, or behavioral analytics.

— **Collaboration with industry and academia**: This would provide the platform for continuously enhancing the IDS. Participation in cyber challenges and sharing what is found will benefit the larger security community.

Data Availability

The dataset associated with this paper has been generated in this research and is publicly available via the Mendeley Elsevier Repository: Large-Scale Network Cyberattacks Multiclass Dataset 2024 (LSNM2024), DOI: 10.17632/7pzyfvv9jn 1. Link: https://data.mendeley.com/datasets/7pzyfvv9jn/1

References

1. Tayyab, U.E.H., Khan, F.B., Durad, M.H., Khan, A., and Lee, Y.S., 2022. A survey of the recent trends in deep learning-based malware detection. The study was published in the Journal of Cybersecurity and Privacy, volume 2, issue 4, on pages 800-829.
2. Townsend, A.M., 2013. Smart cities: Big data, civic hackers, and the quest for a new utopia. W. W. Norton & Company.
3. Sarker, I.H., 2021. Machine learning: Algorithms, real-world applications, and research directions. The article appeared in SN Computer Science, volume 2, issue 3, on page 160.
4. Sarker, I.H., Furhad, M.H., and Nowrozy, R., 2021. AI-driven cybersecurity: an overview, security intelligence modeling, and research directions. The article was published in SN Computer Science, volume 2, on pages 1-18.
5. Olowononi, F.O., Rawat, D.B., and Liu, C., 2020. This survey discusses resilient ML techniques for networked cyber-physical systems, focusing on security measures to protect ML applications in CPS. IEEE Communications Surveys & Tutorials, 23(1), pp. 524-552.
6. N. Ruiz, B. Tavera, and A.-S. Abuzneid, "Intrusion Detection System: The Use of Neural Network Packet Classification," 2020 International Conference on Computational Science and Computational Intelligence (CSCI), Las Vegas, NV, USA, 2020, pp. 1276-1281, doi: 10.1109/CSCI51800.2020.00239
7. X. Liu, "An Abnormal Network Traffic Detection Method on MAWILab Dataset based on Convolutional Neural Network," 2022 *IEEE 2nd International Conference on Electronic Technology, Communication and Information (ICETCI)*, Changchun, China, 2022, pp. 1233–1235, doi: 10.1109/ICETCI55101.2022.9832256.
8. G. J. Pandeeswari and S. Jeyanthi, "Analysis of Intrusion Detection Using Machine Learning Techniques," 2022 Second International Conference on Advanced Technologies in Intelligent Control, Environment, Computing & Communication Engineering (ICATIECE), Bangalore, India, 2022, pp. 1-5, doi: 10.1109/ICATIECE56365.2022.10047057.
9. S. Goel, K. Guleria, and S. N. Panda, "Anomaly-based Intrusion Detection Model using Supervised Machine Learning Techniques," 2022 10th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), Noida, India, 2022, pp. 1-5, doi: 10.1109/ICRITO56286.2022.9965050.
10. B. Sezari, D. P. F. Möller, and A. Deutschmann, "Anomaly-Based Network Intrusion Detection Model Using Deep Learning in Airports," 2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE), New York, NY, USA, 2018, pp. 1725-1729, doi: 10.1109/TrustCom/BigDataSE.2018.00261.
11. F. Rahman, R. Zunaied Tanna, U. Habiba, R. Shaikh, Z. Rahman, and H. Imtiaz, "Cyber Threat Detection Using Machine Learning Algorithms on Heterogeneous MiniVHS-22 Dataset," 2022 25th International Conference on Computer and Information Technology (ICCIT), Cox's Bazar, Bangladesh, 2022, pp. 866-869, doi: 10.1109/ICCIT57492.2022.10055036.
12. A. A. Yilmaz, "Intrusion Detection in Computer Networks using Optimized Machine Learning Algorithms," 2022 3rd International Informatics and Software Engineering Conference (IISEC), Ankara, Turkey, 2022, pp. 1-5, doi: 10.1109/IISEC56263.2022.9998258.
13. B. Adamczyk et al., "Dataset Generation Framework for Evaluation of IoT Linux Host–Based Intrusion Detection Systems," 2022 IEEE International Conference on Big Data (Big Data), Osaka, Japan, 2022, pp. 6179-6187, doi: 10.1109/BigData55660.2022.10020442.
14. Y. Yao, L. Su, and Z. Lu, "DeepGFL: Deep Feature Learning via Graph for Attack Detection on Flow-Based Network Traffic," MILCOM 2018—2018 IEEE Military Communications Conference (MILCOM), Los Angeles, CA, USA, 2018, pp. 579–584, doi: 10.1109/MILCOM.2018.8599821.
15. M. Haggag, M. M. Tantawy, and M. M. S. El-Soudani, "Implementing a deep learning model for intrusion detection on the Apache Spark platform," in IEEE

Access, vol. 8, pp. 163660-163672, 2020, doi: 10.1109/ACCESS.2020.3019931.
16. A. A. Yilmaz, "Intrusion Detection in Computer Networks using Optimized Machine Learning Algorithms," 2022 3rd International Informatics and Software Engineering Conference (IISEC), Ankara, Turkey, 2022, pp. 1-5, doi: 10.1109/IISEC56263.2022.9998258.
17. B. -X. Wang, J. -L. Chen, and C. -L. Yu, "An AI-Powered Network Threat Detection System," in IEEE Access, vol. 10, pp. 54029-54037, 2022, doi: 10.1109/ACCESS.2022.3175886.
18. N. Deeban and P. S. Bharathi, "BRF: Network Traffic Analysis based on Bagging Ensemble with Random Forest Classifier," 2022, 5th International Conference on Contemporary Computing and Informatics (IC3I), Uttar Pradesh, India, 2022, pp. 1920–1925, doi: 10.1109/IC3I56241.2022.10072795.
19. K. Aswal, A. Rajmohan, A. TRC, S. Mukund, V. J. Panicker, and J. P. Dhivvya, "Kavach: A Machine Learning-Based Approach for Enhancing the Attack Detection Capability of Firewalls," 2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kharagpur, India, 2021, pp. 1-5, doi: 10.1109/ICCCNT51525.2021.9579836.
20. K. Shanthi and R. Maruthi, "Machine Learning Approach for Anomaly-Based Intrusion Detection Systems Using Isolation Forest Model and Support Vector Machine," 2023 5th International Conference on Inventive Research in Computing Applications (ICIRCA), Coimbatore, India, 2023, pp. 136-139, doi: 10.1109/ICIRCA57980.2023.10220620.
21. A. Alharthi, A. Eshmawi, A. Kabbas, and L. Hsairi, "Network Traffic Analysis for DDOS Attack Detection". In Proceedings of the 4th International Conference on Future Networks and Distributed Systems (ICFNDS '20). Association for Computing Machinery, New York, NY, USA, 2021, Article 32, pp. 1–6, doi: 10.1145/3440749.3442637.
22. N. Ramakrishnan and T. Soni, "Network Traffic Prediction Using Recurrent Neural Networks," 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), Orlando, FL, USA, 2018, pp. 187-193, doi: 10.1109/ICMLA.2018.00035.
23. A. Srivastava, A. Agarwal, and G. Kaur, "Novel machine learning technique for intrusion detection in recent network-based attacks," 2019 4th International Conference on Information Systems and Computer Networks (ISCON), Mathura, India, 2019, pp. 524-528, doi: 10.1109/ISCON47742.2019.9036172.
24. C.G. Cordero, E. Vasilomanolakis, A. Wainakh, M. Mühlhäuser, and S. Nadjm-Tehrani, "On Generating Network Traffic Datasets with Synthetic Attacks for Intrusion Detection," 2021 ACM Trans. Priv. Secur. 24, 2, Article 8 (May 2021), 39 pages. DOI: 10.1145/3424155.
25. M. Woźniak, J. Siłka, M. Wieczorek, and M. Alrashoud, "Recurrent Neural Network Model for IoT and Networking Malware Threat Detection," in IEEE Transactions on Industrial Informatics, vol. 17, no. 8, pp. 5583-5594, Aug. 2021, doi: 10.1109/TII.2020.3021689.
26. A. Nagaraja, U. Boregowda, and V. Radhakrishna, "Regression analysis for network intrusion detection," 2021 International Conference on Data Science, E-learning and Information Systems (DATA'21). Association for Computing Machinery, New York, NY, USA, pp. 173–179, doi: https://doi.org/10.1145/3460620.3460751.
27. K.V. Nguyen, H.T. Nguyen, T.Q. Le, and Q.N. Truong, "Abnormal network packets identification using header information collected from Honeywall architecture," 2023 Journal of Information and Telecommunication, 7:4, pp. 437-461, doi: 10.1080/24751839.2023.2215135.
28. A. Abid, F. Jemili, and O. Korbaa, "Distributed deep learning approach for intrusion detection system in industrial control systems based on big data technique and transfer learning," Journal of Information and Telecommunication, 7:4, pp. 513-541, doi: 10.1080/24751839.2023.2239617.
29. S. Dalal, U. Lilhore, N. Faujdar, S. Simaiya, M. Ayadi, N. Almujally, and A. Ksibi wrote an article titled "Next-generation cyber-attack prediction for IoT systems: using multi-class SVM and improved CHAID decision tree," published in 2023 in the Journal of Cloud Computing: Advances, Systems and Applications, volume 12, issue 1
30. De Silva, K.N.H., Manchanayaka, M.A.S.B., Punyasiri, D.L.S.I., Perera, H.A.D.N., Gamage, A., and Gamage, N.," Real-time Network-Based Anomaly Detection and Malware Analysis for SMEs and Smart Homes," International Research Journal of Innovations in Engineering and Technology, vol. 7, no. 10, p. p.249, doi: 10.47001/IRJIET/2023.710032.
31. K.M. Abuali, L. Nissirat, A. Al-Samawi, "Advancing Network Security with AI: SVM-Based Deep Learning for Intrusion Detection," Sensors 2023, vol. 23, p. 8959, doi: 10.3390/s23218959.
32. D. Santhadevi and B. Janet, "HSDL-based intelligent threat detection framework for IoT network," J. Intell. Fuzzy Syst. Vol. 45, no. 1 (2023), pp. 1775–1790, doi: 10.3233/JIFS-223246.
33. R. Abu Bakar, B. Kijsirikul, "Enhancing Network Visibility and Security with Advanced Port Scanning Techniques," Sensors 2023, vol. 23, p. 7541, doi: 10.3390/s23177541.
34. M.M. Wrana, M. Elsayed, K. Lounis, Z. Mansour, S. Ding, and M. Zulkernine wrote an article titled "OD1NF1ST: True Skip Intrusion Detection and Avionics Network Cyber-attack Simulation," which was published in the October 2022 issue of the 2022 ACM Transactions on Cyber-Physical Systems.
35. M. Rawashdeh, Q. A. Al-Haija, and M. Qasaimeh, "Analysis of TOR Artifacts and Traffic in Windows 11: A Virtual Lab Approach and Dataset Creation," 2023, 14th International Conference on Information and Communication Systems (ICICS), Irbid, Jordan, pp. 1–6, doi: 10.1109/ICICS60529.2023.10330539.
36. Q.A. Al-Haija, M. Al-Fayoumi, "An intelligent identification and classification system for malicious uniform resource locators (URLs)," Neural Comput & Applic. vol. 35, pp. 16995–17011 (2023), doi: 10.1007/s00521-023-08592-z.
37. A. Garg and P. Maheshwari, "Performance analysis of a Snort-based Intrusion Detection System," 2016 3rd International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2016, pp. 1–5, doi: 10.1109/ICACCS.2016.7586351.
38. Z. Zhou, Z. Chen, T. Zhou, and X. Guan, "The study on the network intrusion detection system of Snort," 2010 International Conference on Networking and Digital Society, Wenzhou, China, 2010, pp. 194–196, doi: 10.1109/ICNDS.2010.5479341.
39. A. Tiwari, S. Saraswat, U. Dixit, and S. Pandey, "Refinements in Zeek Intrusion Detection System," 2022 8th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2022, pp. 974-979, doi: 10.1109/ICACCS54159.2022.9785047.
40. Ahmad, Israr, Munam Ali Shah, Hasan Ali Khattak, Zoobia Ameer, Murad Khan, and Kijun Han. 2020. "FIViz: Forensics Investigation through Visualization for Malware in Internet of Things." Sustainability 12, no. 18: 7262. https://doi.org/10.3390/su12187262
41. Cermak, M., 2021. Graph-based Network Traffic Analysis for Incident Investigation.
42. S. Sengupta, A. Chowdhary, A. Sabur, A. Alshamrani, D. Huang, and S. Kambhampati, "A Survey of Moving Target Defenses for Network Security," in IEEE Communications Surveys & Tutorials, vol. 22, no. 3, pp. 1909-1941, third quarter 2020, doi: 10.1109/COMST.2020.2982955.
43. Mathijssen, M., Overeem, M., and Jansen, S., 2020. Identification of practices and capabilities in API management: a systematic literature review. arXiv preprint arXiv:2006.10481, doi.org/10.48550/arXiv.2006.10481
44. S. Suthaharan and T. Panchagnula, "Relevance feature selection with data cleaning for intrusion detection system," 2012 Proceedings of IEEE Southeastcon, Orlando, FL, USA, 2012, pp. 1-6, doi: 10.1109/SECon.2012.6196965.